The Synthesis Prototype:

Derivation Replacing Authorship in a Bounded, Receipted Pipeline

semi-naive saturation → constraint-discovered sequencing → content-addressed execution → refinement admission

Sean Chatman

with the Praxis working system, `crates/praxis-synthesis` v26.7.2

2 July 2026 — Genesis, Day 9

Abstract

On Genesis Day 2 a five-step plan — *supply-evidence, clear-obligations, judge, admit, receipt* — was written by hand, in PDDL, by a person who knew the answer. This thesis describes a working prototype, `praxis-synthesis`, in which the same plan is *derived*: from declared capabilities and a saturated fact base, a bounded solver discovers both the order and the parameter bindings, executes the result as a content-addressed DAG whose second run replays entirely from cache, and admits the whole artifact through machine-checkable refinements — including an independent replay that does not trust the solver's own bookkeeping. Each of the four layers is the local instantiation of a published result (Nemo's scalable Datalog; SMT-discovered capability sequencing; OxyMake's content-addressable workflows; Flux's refinement discipline), and each rides on a substrate the host system already owned. We derive the pipeline from first principles — the least fixpoint algebra of saturation, the counting argument for semi-naive evaluation, the search calculus of bounded sequencing, and the collision-bounded geometry of content addressing — and we keep the ledger honest: every claim below is labeled *theorem* (proved), *measurement* (observed in the test suite), or *refusal* (deliberately not done, with reasons). The central thesis: **when capabilities are declared rather than plans authored, correctness moves from the author's head into the derivation chain — and the derivation chain, unlike the head, can be hashed.**

Contents

1 The Problem: Authorship Does Not Scale

A capability-driven system — praxis, and by intent any fleet of lawful agents — must repeatedly answer one question: *given what is true and what the system can do, what sequence of actions lawfully reaches the goal?*

There are two ways to answer it.

Authorship. A person (or an LLM prompted by a person) writes the plan: a PDDL problem, a workflow file, a runbook. Genesis Day 2 did exactly this — the five-step lawobject pipe was hand-encoded, and it was correct because its author understood the domain. Authorship has a hidden cost function: it is linear in the number of *plans*, and the number of plans grows with the product of goals, initial states, and domain variants. A trillion-agent fleet does not have a trillion authors.

Derivation. The system is told only what is true (facts), what follows from truth (rules), and what each capability needs and produces (declarations). The plan is then a *computed object*, and its correctness is a property of the derivation procedure — provable once, reused for every instance. Derivation's cost is paid in machinery: a reasoner to close the facts, a solver to discover the sequence, an executor whose outputs can be audited without rerunning it, and a verifier that does not take the solver's word for anything.

The prototype under study builds precisely that machinery, under two standing constraints inherited from the host system's doctrine:

1. **Bounded everything.** Every loop has a cap; every cap violation is a first-class refusal carrying salvage data. No panic, no silent truncation.
2. **Receipted everything.** Every stage emits a content address; the run composes them into a single hash. Trust attaches to the chain, not to the narrative.

Four published results, surfaced by a verified research sweep on 2 July 2026, supply the blueprint: Nemo demonstrates that Datalog saturation over 10^5–10^8 facts is a laptop-scale operation [1]; the semantic capability-model literature shows solvers can discover execution orders and parameter bindings that engineers currently hand-encode [2]; OxyMake demonstrates content-addressed workflow outputs that make reproducibility a property of hashes rather than of execution order [3]; and Flux shows that lightweight refinements — predicates carried alongside artifacts — catch the classes of error that full verification is too expensive to reach [4]. The prototype is the composition of the four lessons in 1,800 lines of bounded Rust.

2 Layer 1 — Saturation: the Algebra of What Follows

2.1 The immediate-consequence operator

Fix a finite alphabet of interned symbols. The prototype inherits its interning from the host's grounding crate: a dictionary $\delta : \Sigma \to \{0, \ldots, n-1\} \subset \mathbb{N}$ maps strings to dense `u32` identifiers, and a ground atom is a pair $(p, \bar{a})$ with p a predicate id and $\bar{a}$ an argument tuple. A *database* $\mathcal{D}$ is a finite set of ground atoms.

Definition 2.1 (Rule, safety)**.** A rule r is $h \leftarrow b_1, \ldots, b_k,\ \neg m_1, \ldots, \neg m_j$ where h, b_i, m_i are atoms over variables and constants, subject to the *safety invariant*: every variable occurring in h or in any m_i occurs in some positive b_i. The prototype enforces this at `add_rule` and refuses unsafe rules with a reasoned error.

Definition 2.2 (Immediate consequence)**.** For a program P (a finite rule set) define $T_P : 2^{\mathsf{Atoms}} \to 2^{\mathsf{Atoms}}$ by

$$T_P(\mathcal{D}) \;=\; \mathcal{D} \cup \Big\{\, h\theta \;\Big|\; r \in P,\ \theta \text{ grounds } r,\ b_i\theta \in \mathcal{D}\ \forall i,\ m_i\theta \notin \mathcal{D}^-\ \forall i \,\Big\},$$

where $\mathcal{D}^-$ denotes the (already-saturated) database of strictly lower strata — the stratification condition defined below.

Theorem 2.3 (Existence and finiteness of the fixpoint)**.** *For a stratified program over a finite symbol universe, within each stratum T_P is monotone on the complete lattice $(2^{\mathsf{Atoms}}, \subseteq)$, hence by Knaster–Tarski has a least fixpoint $\mathrm{lfp}(T_P) = \bigcup_{i \geq 0} T_P^i(\mathcal{D}_0)$; and because the ground-atom universe is finite (at most $|\mathsf{preds}| \cdot n^{\mathsf{arity}}$ atoms), the ascending chain stabilizes after finitely many steps.*

Proof. Monotonicity: within a stratum the negated atoms are evaluated against the *frozen* lower strata $\mathcal{D}^-$, not against the growing $\mathcal{D}$, so enlarging $\mathcal{D}$ can only enlarge the set of satisfied positive premises. Knaster–Tarski then gives the least fixpoint as the limit of the ascending Kleene chain. Finiteness of the universe bounds the chain length by its cardinality. □

Stratification itself is computed by the standard relaxation: assign $\mathrm{str}(h) \geq \mathrm{str}(b_i)$ and $\mathrm{str}(h) > \mathrm{str}(m_i)$, iterate to fixpoint, and refuse (Section 7) if the required stratum ever reaches the cap — which happens exactly when negation cycles through a predicate's own definition, the classic *win/lose* pathology, and which the test suite pins as a receipted `Unstratifiable` refusal rather than an infinite loop.

2.2 Why semi-naive: the counting argument

The naive Kleene iteration recomputes every join against the *whole* database each round. The prototype implements *semi-naive* evaluation: after the first round, each rule is re-joined once per positive body position, with that position restricted to the tuples derived in the previous round (the delta).

Proposition 2.4 (Semi-naive soundness and completeness)**.** *Every atom derivable by naive iteration is derived by semi-naive iteration, and conversely.*

Proof sketch. Soundness is immediate (semi-naive fires a subset of naive instantiations). Completeness: any *new* derivation at round $i{+}1$ must use at least one premise derived at round i (else it fired at round i already); the per-position delta restriction enumerates every choice of which premise is new. Duplicate firings across positions are harmless because insertion is idempotent. □

Proposition 2.5 (Work bound)**.** *Let Δ_i be the atoms first derived at round i and F the final database. Naive evaluation performs $\Theta\big(\sum_i |F|^k\big)$ pattern probes in the worst case (k the maximal body length), while semi-naive performs $O\big(\sum_i k\,|\Delta_i| \cdot |F|^{k-1}\big)$: each round's work is proportional to* what changed*, not to* what is known*.*

This is the entire content of the Nemo lesson at prototype scale: saturation cost must track novelty. The measured witness:

Measurement 2.6 (Transitive closure, 100-node chain)*.* With *edge* facts forming a 100-node path and the two-rule transitive-closure program, saturation derives exactly $\binom{100}{2} = 4{,}950$ *path* atoms; the test additionally asserts multi-round convergence and endpoint reachability $path(n_0, n_{99})$. The identical program under the host's backward-chaining kernel is out of reach by construction: that kernel's derivation depth is capped at 32, while the chain requires depth 99. Test: `transitive_closure_over_100_node_chain_saturates`, green.

2.3 The fixpoint as a value: hashing the reasoned state

Saturation ends by computing

$$\mathsf{fixpoint\_hash}(\mathcal{D}) = \mathsf{H}\Big(\ \big\|_{k \in \mathrm{sort}(K(\mathcal{D}))} k\ \Big), \qquad K(\mathcal{D}) = \{\, \mathsf{key}(p, \bar{a}) \mid (p, \bar{a}) \in \mathcal{D} \,\}$$

where key is the host's 64-bit atom packing and H is BLAKE3. Sorting makes the hash a function of the *set*, not of derivation order.

Proposition 2.7 (Extensional identity)**.** *Two saturations agree on* $\mathsf{fixpoint\_hash}$ *iff they computed the same set of packed atom keys (up to* H *collision and 64-bit key collision, both addressed in Section 4.3).*

The consequence is operationally important: *the reasoned state is now a value.* Layer 2 hashes it into the problem statement; the pipeline hashes it into the run receipt; a verifier can check that two agents reasoned from the same closure without exchanging the closure. The test suite pins both directions — identical inputs give identical hashes, and one extra fact changes the hash.

3 Layer 2 — Sequencing: Derivation Replaces Authorship

3.1 Capabilities as declarations

Definition 3.1 (Capability)**.** A capability is a tuple $c = (\mathsf{name}, k, \mathsf{pre}, \mathsf{add}, \mathsf{del}, \mathsf{cost})$ with k parameter variables and pattern-atom lists over those variables, subject to the same safety invariant as rules: every variable in $\mathsf{add} \cup \mathsf{del}$ is bound by pre. Violations are refused at problem construction.

A *sequencing problem* is $(\mathcal{C}, \mathcal{D}_0, \gamma, H)$: capabilities, the Layer-1 saturated database as initial state, goal patterns γ, horizon H. Note what is absent: there is no plan, no ordering, no PDDL text. The declaration says what each capability *is*; the solver decides what to *do*.

3.2 The search calculus and its bounds

The bundled solver is deterministic branch-and-bound depth-first search over bound steps:

$$\mathrm{state} \xrightarrow{(c,\theta)} \big(\mathrm{state} \setminus \mathsf{del}\,\theta\big) \cup \mathsf{add}\,\theta,$$

where at each depth the candidate bindings θ are enumerated by *joining* $\mathsf{pre}(c)$ *against the current state* — the same join algebra as Layer 1. This is the architectural point the research sweep recommended and the prototype realizes: **the reasoner's closure is the solver's binding source.** A capability whose precondition mentions a *derived* predicate (one produced by rules, never asserted) is sequencable with no extra machinery, because saturation already materialized it.

Proposition 3.2 (Soundness of discovered plans)**.** *If the solver returns $\pi = \langle (c_1, \theta_1), \ldots, (c_m, \theta_m) \rangle$, then executing π from $\mathcal{D}_0$ under the transition relation above yields a state satisfying γ, and $m \leq H$.*

Proof. The DFS applies exactly the transition relation while descending and restores state exactly while backtracking (the implementation tracks the precise added/removed sets per step, so undo is exact even when effects overlap). A plan is recorded only at a node whose state satisfies the goal join; depth never exceeds H by the guard. □

Proposition 3.3 (Optimality within the horizon)**.** *Among plans of length $\leq H$ reachable under the caps, the returned plan minimizes total cost: branch-and-bound prunes only branches whose accumulated cost already meets or exceeds the incumbent, which cannot exclude a strictly cheaper completion because costs are non-negative.*

Proposition 3.4 (Determinism)**.** *Candidate capabilities are tried in declaration order and bindings in the sorted order of the state's tuple storage; therefore identical problems produce byte-identical plans and receipts. (Pinned by test: two full solves serialize identically.)*

The caps are explicit constants: horizon ≤ 16, bindings per step ≤ 256, search nodes $\leq 10^5$. Exhaustion is not an error state but a *receipt*: the refusal carries the node count and the best plan found so far — salvage, in the doctrine's vocabulary.

3.3 The flagship derivation

Measurement 3.5 (The lawobject pipe, rediscovered). Declare the five capabilities *supply-evidence* ($\mathit{raw}\, x \Rightarrow \mathit{evidence}\, x$), *clear-obligations* ($\mathit{evidence}\, x \Rightarrow \mathit{clear}\, x$), *judge* ($\mathit{clear}\, x \Rightarrow \mathit{validated}\, x$), *admit* ($\mathit{validated}\, x \Rightarrow \mathit{admitted}\, x$), *receipt* ($\mathit{admitted}\, x \Rightarrow \mathit{receipted}\, x$), each of cost 1; assert $\mathit{raw}(o_1)$; goal $\mathit{receipted}(o_1)$; horizon 6. The solver returns exactly

$$\langle \text{supply-evidence},\ \text{clear-obligations},\ \text{judge},\ \text{admit},\ \text{receipt} \rangle, \quad \theta_i = [x \mapsto o_1]\ \forall i, \quad \text{cost } 5,$$

i.e. the order and the binding that Genesis Day 2 hand-authored in PDDL — now derived from declarations alone. An independent replay (not the solver's bookkeeping) re-executes the plan and confirms the goal. Tests: `solver_rediscovers_the_five_step_lawobject_order`; unsatisfiable goals and too-short horizons refuse with reasoned `Unsatisfiable`; all green.

Remark 3.6 (What this is and is not). The example is deliberately small — a chain, not a lattice. Its value is not combinatorial difficulty but *provenance*: an artifact that previously existed only because a human wrote it now exists because a derivation produced it, and the derivation is hashed into the problem receipt. The refusal register (Section 7) is explicit that real SMT theories — arithmetic constraints, temporal windows, resource envelopes — are the seam left open, behind a trait.

4 Layer 3 — Execution: the Geometry of Content Addressing

4.1 From linear plan to DAG

A discovered plan is a sequence, but its causal structure is sparser. Define the *data-dependency graph* $G(\pi)$: node i per bound step, edge $j \to i$ iff some ground precondition atom of step i

was most recently produced by an add-effect of step j ($j < i$). For the lawobject pipe this recovers the pure chain (4 edges over 5 nodes — pinned by test); for two steps on disjoint objects it produces no edge at all, and the concurrency that the linear plan hid becomes visible structure.

Proposition 4.1 (Acyclicity by construction)**.** *$G(\pi)$ is acyclic: every edge points from an earlier to a later plan index. (Hand-built graphs can cycle; the executor's Kahn ordering therefore re-checks and refuses cycles — also pinned by test.)*

4.2 Execution as hashing

Execution walks a deterministic topological order (Kahn's algorithm, ties broken by node identity). For node v with action hash a_v and sorted input output-hashes $\bar{h}_v$:

$$\mathsf{key}(v) = \mathsf{ca}(a_v \,\|\, \bar{h}_v), \qquad \mathsf{out}(v) = \begin{cases} \text{memo}[\mathsf{key}(v)] & \text{if present (replay)} \\ \text{runner}(v, \text{inputs}) & \text{otherwise (compute, then memoize)} \end{cases}$$

and the node receipt folds $\mathsf{chain} \leftarrow \mathsf{fold}(\mathsf{chain}, \{v, a_v, \bar{h}_v, \mathsf{H}(\mathsf{out}(v))\})$ — the host's $h^+ = \mathsf{H}(h^- \| \mathsf{frame})$ discipline, seeded from a domain constant.

Two hashes with different jobs close the layer:

- the **chain** (order-sensitive) proves *this execution happened in this order and nothing was altered after the fact* — the test suite tampers with one recorded output hash and shows the refold no longer matches;
- the **root** (order-insensitive), H over the sorted (id : outhash) pairs, names *the executed artifact set itself*, independent of scheduling.

Theorem 4.2 (Reproducibility is content-addressed, not order-addressed)**.** *If the runner is a function of $(action, inputs)$, then (i) node identity being content-based (hash of capability, binding, occurrence) makes* root *invariant under any re-declaration order that yields the same bound steps, and (ii) a second execution against a warm memo cache computes zero nodes and reproduces the identical root and chain.*

Proof, and its witnesses. (i) Node ids, edges, keys, and outputs are all functions of step content, and the root hashes a *sorted* set. Witness: the two-independent-steps test builds the problem in both declaration orders and asserts equal roots. (ii) Every memo key present $\Rightarrow$ every node replays; the fold consumes identical frames. Witness: the warm-cache pipeline test asserts $\mathsf{replayed} = |V|$ and byte-equal chains. Both green. □

This is the OxyMake lesson in one sentence: *an output is named by what made it, so recomputation is a cache lookup and reproduction is an equality check.*

4.3 Collision honesty

All identity claims are "up to hash collision." Two regimes matter. BLAKE3 at 256 bits: for N hashed objects the collision probability is $\leq N^2/2^{257}$; at a generous $N = 10^{12}$ lifetime objects this is $< 10^{-53}$ — ignorable against any physical failure mode. The 64-bit *atom-key packing* inherited from the host is the honest caveat: at $N = 10^5$ atoms the birthday bound gives collision probability $\approx N^2/2^{65} \approx 2.7 \times 10^{-10}$; at Nemo-scale $N = 10^8$ it reaches $\approx 2.7 \times 10^{-4}$ and the packing must widen before the scalability claim transfers. This is recorded as a refusal (Section 7), not hidden. These are *estimates*, not theorems: they assume uniform hashing.

5 Layer 4 — Admission: the Verifier Does Not Trust the Solver

The Flux lesson, transposed from types to artifacts: attach to each artifact a *refinement* — a machine-checkable predicate whose evidence travels with the object. The prototype's verdict runs six refinements with no short-circuit (every check reports, in the host's `run_pipeline` style):

Refinement	Checks	Independence
PlanRespectsHorizon	$\|\pi\| \leq H$	syntactic
PlanReachesGoal	re-executes π from $\mathcal{D}_0$	**does not consult solver state**
DagAcyclic	Kahn orders all $\|V\|$ nodes	structural
EveryNodeReceipted	$\|\text{receipts}\| = \|V\|$	structural
ChainRecomputes	refolds every frame, byte-compares	**recomputation, not inspection**
FixpointClosed	one more round derives nothing	**re-runs the reasoner**

Proposition 5.1 (The differential stance)**.** *PlanReachesGoal is a second implementation of plan semantics (the problem's replay path), and ChainRecomputes is a second computation of the chain. A solver bug that records a wrong plan, or an executor bug that mis-folds a frame, is caught by disagreement between independent computations — the same differential method that caught four real bugs elsewhere in the host system during Genesis week.*

Admission is not advisory: `Synthesis::run` *refuses* the entire run (with the failed refinement names as salvage) if any check fails. A plan that cannot survive its own replay does not become an artifact.

6 The Composition: One Run, One Hash

The pipeline threads the four layers and folds their stage hashes:

$$\mathsf{chain}_{\text{run}} = \mathsf{fold}\big(\cdots \mathsf{fold}\big(\mathsf{fold}\big(\mathsf{fold}(g,\ \mathsf{fixpoint\_hash}),\ \mathsf{plan\_hash}\big),\ \mathsf{root\_hash}\big),\ \mathsf{verdict}\big)$$

seeded from the pipeline domain constant g. The resulting `SynthesisReceipt` is a single value that commits, in order: what was known (after reasoning), what was decided, what was done, and whether it survived admission.

Measurement 6.1 (End-to-end). On the lawobject domain: 5 steps discovered, 6/6 refinements pass, the whole-run chain is a single 64-hex BLAKE3 value; the run is a pure function of its input (two runs byte-agree on all four stage hashes and the chain); and a warm-cache rerun replays 100% of nodes yet produces the *identical* receipt. Whole crate: **20/20 tests green, clippy 0 warnings, full workspace `--all-features` exit 0.**

Remark 6.2 (Relation to the Chatman equation). In the host doctrine's terms $A = \mu(O^*)$: the admitted observation O^* is here the saturated, hashed fact base; the manufacturing morphism μ is the solver-plus-executor; and $R = \text{receipt}(A)$ is the composed chain. The prototype's specific contribution is to make μ itself derived — the morphism is discovered by search over declarations, not encoded by an author — while keeping R's discipline intact.

7 The Refusal Register

Silent gaps are the only forbidden artifact. What this prototype deliberately does not do, each with reason and salvage:

Refusal 7.1 (No native SMT backend). Z3/cvc5 are native libraries; linking them breaks the pure-Rust, forbid-unsafe, deterministic-build doctrine. *Salvage:* the Solver trait is the seam; BoundedCsp proves the declare-and-derive architecture with theories deferred, not denied.

Refusal 7.2 (No Nemo-scale benchmarks). The join is nested-loop over sorted relations; the claim proved is architectural (novelty-proportional saturation), demonstrated at 10^3–10^5 tuples, not 10^8. *Salvage:* storage is already the interned sorted-ID form that index joins and worst-case-optimal joins consume; and the 64-bit atom key must widen first (Section 4.3).

Refusal 7.3 (No parallel execution; no persistent memo cache). Prototype scope: determinism first. *Salvage:* content addressing already makes disjoint-input nodes safely parallelizable, and the memo cache is a plain map behind a stable key scheme.

Refusal 7.4 (Data edges only in the DAG). Delete-effects do not order otherwise-independent nodes; a delete/precondition conflict between concurrent branches is representable but not yet prevented by an edge. *Salvage:* the discovered plans' linear order is never violated by from_plan; conservative interference edges are a local addition.

Refusal 7.5 (No CLI verb, no membrane tool). Surface stability: the API should survive contact before becoming a public noun. *Salvage:* `ops::`-mediated wiring is one thin function per verb, the pattern nine existing nouns already follow.

8 Conclusion: What Moved

Genesis Day 2's plan was correct because a person understood the domain. Day 9's identical plan is correct because a fixpoint was proved to exist (Theorem 2.3), a search was proved sound within its caps (Proposition 3.2), an execution was made reproducible by construction (Theorem 4.2), and an admission layer recomputed everything it was told (the differential stance) — with every one of those claims either proved above, measured in a green test, or refused on the record.

That is the thesis in one movement: **authorship placed correctness in a head; derivation places it in a chain.** Heads do not scale and cannot be hashed. Chains do, and can.

References

[1] A. Ivliev et al. *Nemo: First Glimpse of a New Rule Engine.* ICLP 2023 system demonstration; arXiv:2308.15897. Datalog saturation over 10^5–10^8 facts on commodity hardware.

[2] A. Köcher et al. *Automating the Generation of Planning Problems from Semantic Capability Models.* arXiv:2312.08801. Solver-discovered capability sequences and parameter bindings from declared capability models.

[3] *OxyMake: Content-Addressable Workflow Automation in Rust.* arXiv:2606.20989 (2025). BLAKE3-named outputs; reproducibility independent of execution order.

[4] N. Lehmann, A. Geller, N. Vazou, R. Jhala. *Flux: Liquid Types for Rust.* PLDI 2023; arXiv:2207.04034. Lightweight refinements integrated with ownership.

[5] A. Tarski. *A lattice-theoretical fixpoint theorem and its applications.* Pacific J. Math. 5 (1955) 285–309.

[6] S. Abiteboul, R. Hull, V. Vianu. *Foundations of Databases.* Addison–Wesley, 1995. Chapters 12–15: Datalog, semi-naive evaluation, stratified negation.

[7] J. O'Connor, J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn. *BLAKE3: One Function, Fast Everywhere*. 2020.

[8] S. Chatman. *Beyond Cognitive Load, Within Admissible Projection*. Praxis, `docs/thesis/projection_thesis.pdf`, Genesis Day 1 (2026).